

Don't Trust All Your Neighbors: Re-implementing Graph Attention Networks

Yash Chatha (yc2727) Jerry Ji (rj378) Leon Huang (lyh7)
CS 4782: Deep Learning, Cornell University, Spring 2026

1. Introduction

Graph neural networks address node classification on graph-structured data by aggregating information from a node's local neighborhood. This project re-implements **Graph Attention Networks (GAT)** by Veličković et al. (ICLR 2018). The paper's primary contribution is replacing the fixed equal-weight neighbor aggregation of Graph Convolutional Networks (GCN) with a learned attention mechanism: for each node, the model computes a separate attention score per edge, weighting neighbors by relevance rather than treating them uniformly. This is achieved without requiring knowledge of the full graph structure upfront.

2. Chosen Result

We reproduce **Table 2**: transductive node classification accuracy on Cora and CiteSeer averaged over 100 runs. The paper reports $83.0 \pm 0.7\%$ on Cora and $72.5 \pm 0.7\%$ on CiteSeer; this is the central claim that learned attention outperforms fixed-weight GCN aggregation ($81.5\% / 70.3\%$). Reproducing it validates the architecture and training protocol, and establishes a baseline for our extensions.

3. Methodology

Architecture. We implement a 2-layer GAT using PyTorch Geometric's `GATConv`. Layer 1 uses $K = 8$ attention heads each computing $F' = 8$ features, concatenated to a 64-dimensional node representation with ELU activation. Layer 2 uses a single attention head outputting C class logits followed by log-softmax. Dropout $p = 0.6$ is applied to both layer inputs and attention coefficients, matching the paper's regularization exactly.

Training. Adam optimizer, $lr = 0.005$, L2 weight decay $\lambda = 0.0005$, up to 10,000 epochs with early stopping (patience 100). **Modification:** we monitor only validation loss for early stopping, rather than both validation loss and accuracy as in the original TensorFlow implementation. This was a deliberate choice to ensure a consistent, comparable training protocol across all experiments, particularly the low-label benchmark where label scarcity introduces noise that makes accuracy-based stopping less stable.

Datasets. Cora (2,708 nodes, 1,433 features, 7 classes) and CiteSeer (3,327 nodes, 3,703 features, 6 classes), loaded via PyG Planetoid with `NormalizeFeatures`. Standard public splits: 20 nodes/class train, 500 val, 1,000 test.

Evaluation. 100 seeds; mean $\pm$ std test accuracy reported.

Extensions. Beyond reproduction, we designed three independent experiments: (1) a *low-label benchmark* varying the fraction of training labels from 10% to 100%; (2) *TinyGAT distillation*, training a $6\times$ smaller student model using the full GAT as a knowledge distillation teacher; and (3) a *failure case explorer*, analyzing every misclassified test node's confidence, neighbor class distribution, and per-edge attention weights.

4. Results & Analysis

Baseline reproduction.

Dataset	Paper	Ours	Gap
Cora	$83.0 \pm 0.7\%$	$83.22 \pm 0.41\%$	$+0.22\%$
CiteSeer	$72.5 \pm 0.7\%$	$70.94 \pm 0.52\%$	-1.56%

Cora matches and slightly exceeds the paper’s target. The CiteSeer gap is attributable to our early stopping deviation: monitoring loss only is a less sensitive stopping signal than loss-plus-accuracy, which may cause the model to stop at a suboptimal point on CiteSeer’s harder optimization landscape.

Low-label benchmark. Accuracy degrades monotonically as labels decrease, but the magnitude differs sharply. On Cora, performance at 10% labels (14 nodes) is $58.8 \pm 5.9\%$, well above random chance (14.3%). On CiteSeer, the same fraction (12 nodes) collapses to $30.2 \pm 17.9\%$, with one seed hitting 8.3%, below random chance (16.7%). We attribute this to CiteSeer’s sparser feature vocabulary (3,703-dim with few non-zero entries per node), leaving the model with negligible discriminative signal at extreme label scarcity.

TinyGAT distillation. TinyGAT (2 heads $\times$ 4 features, $\sim 6\times$ fewer parameters) trained with KL-divergence soft targets (temperature=2.0, $\alpha = 0.7$) showed mean distillation gain of -0.24% on Cora and -0.66% on CiteSeer. TinyGAT with supervision alone already achieves 82.5% / 70.8%, nearly matching the teacher. Model capacity is not the bottleneck; both models saturate the available signal.

Failure case analysis. We analyzed 25 Cora and 24 CiteSeer misclassifications for seed 1. Two failure modes emerged: (1) *ambiguous neighborhoods*, where nodes with mixed-class neighbors receive nearly uniform attention weights, producing low-confidence errors that may be irreducible; and (2) *hub node influence*, where a single high-attention wrong-class neighbor dominates aggregation. On Cora, node 1358 (class 2) appears in the top-5 attention sources for 6 of 25 failures, with two errors exceeding 92% confidence. Since attention weights are lower-bounded at zero (post-softmax), the model can reduce but never cancel a misleading neighbor’s contribution.

5. Reflections

Key lessons. Hyperparameter choices interact non-trivially with dataset difficulty: our early stopping deviation (loss-only) costs approximately 1.5% on CiteSeer but is negligible on Cora. Future re-implementations should match the exact stopping criterion before declaring a gap.

Surprising findings. CiteSeer’s collapse to near-random at 10% labels was unexpected; standard benchmark results do not expose this feature-sparsity sensitivity. The distillation null result reframes Cora and CiteSeer as capacity-saturated benchmarks where architecture above a low threshold doesn’t matter.

Future directions. (1) Run low-label and distillation experiments on PubMed and PPI to test generalization at scale; (2) explore repulsive attention mechanisms that can down-weight misleading hub neighbors beyond what softmax allows; (3) ablate CiteSeer’s low-label collapse by independently controlling feature density and graph topology.

References

- [1] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph Attention Networks. *ICLR 2018*.
- [2] Fey, M. & Lenssen, J.E. (2019). Fast Graph Representation Learning with PyTorch Geometric. *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- [3] Yang, Z., Cohen, W., & Salakhutdinov, R. (2016). Revisiting Semi-Supervised Learning with Graph Embeddings. *ICML 2016*.
- [4] Kipf, T.N. & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. *ICLR 2017*.